

Java Developer Hiring Assignment – CRDT Offline Sync POC

Purpose of This Assignment

This assignment is part of the technical evaluation process. The objective is to assess the candidate's ability to design, reason about, and implement distributed-system logic using Java. Candidates who successfully complete this task will proceed to the hiring stage.

Problem Statement

Design and implement a Java-based Proof of Concept (POC) for an offline-first collaborative text system. Multiple clients should be able to modify a shared document independently (offline) and later synchronize changes without conflicts.

Key Challenge (Mandatory)

The system must correctly handle out-of-order operations, specifically the case where a DELETE operation arrives before the corresponding INSERT operation (delete-before-insert). This scenario must not cause data inconsistency.

Functional Requirements

- 1 Use pure Java (no Spring Boot, no external CRDT libraries).
- 2 Support INSERT and DELETE operations using an operation-log approach.
- 3 Each operation must have a globally unique identifier (Lamport timestamp + clientId + sequence).
- 4 INSERT operations must be anchor-based (not index-based).
- 5 DELETE operations must use tombstones (no hard deletes).
- 6 DELETE-before-INSERT must be handled correctly using a pending-tombstone or equivalent mechanism.
- 7 Provide delta synchronization using a version or offset mechanism.
- 8 Final document must be deterministically rendered from operations.

Required API Endpoints

- 1 POST /op – Submit an INSERT or DELETE operation.
- 2 GET /sync?since=N – Fetch operations newer than version N.
- 3 GET /doc – Return the current rendered document.
- 4 GET /health – Basic server health and state.
- 5 GET /demo – Demonstrate delete-before-insert scenario.

Delete-Before-Insert Demo Requirement

Your solution must include a reproducible demo or test case where: 1. A DELETE operation is processed before the INSERT operation it targets. 2. The INSERT operation arrives later. 3. The deleted element never appears in the final document. 4. Subsequent INSERT operations referencing the deleted element still work correctly. Expected final document output: "i"

Non-Functional Requirements

- 1 Code must be clean, readable, and well-structured.
- 2 Proper naming conventions and inline comments are expected.

- 3 Solution must be deterministic and repeatable.
- 4 No UI is required; API or console output is sufficient.

Deliverables

- 1 Runnable Java source code (single file or small project).
- 2 README explaining how to compile, run, and test the solution.
- 3 Curl commands or steps to verify API behavior.
- 4 Explanation (in README or comments) of how delete-before-insert is handled.

Evaluation Criteria

- 1 Correctness of CRDT logic and conflict handling.
- 2 Ability to reason about distributed systems problems.
- 3 Clean and maintainable Java code.
- 4 Clear explanation of design decisions.
- 5 Successful handling of delete-before-insert scenario.

Outcome

Candidates who successfully complete this assignment and demonstrate a clear understanding of the design decisions will be considered for the next stage of the hiring process.

Good luck, and we look forward to reviewing your solution.